

Embedded OS Implementation, Spring 2025

Project #2 (due May 8, 2025 (Thursday) 8:00)

[PART I] EDF Scheduler Implementation

Objective:

To implement the Earliest-Deadline-First (EDF) scheduler for periodic tasks and to observe the scheduling behaviors.

Problem Definition:

uC/OS-II supports priority-driven scheduling. However, it lacks deadline-driven scheduling. In this assignment, you are going to implement the EDF scheduler in uC/OS-II. To accomplish this assignment, you must know about the scheduler of uC/OS-II. It can be implemented based on the existing data structures of uC/OS-II. The objectives of this assignment are the following:

- (1) To add some functional data structures for your EDF scheduler.
- (2) To cooperate with existing data structures/mechanisms in uC/OS-II.

Implement the following examples. Add necessary code to the μ C/OS-II scheduler in the kernel level to observe how the task suffers the schedule delay.

Periodic Task Set = $\{\tau_{ID} (ID, \text{arrival time}, \text{execution time}, \text{period})\}$

Example Task Set 1 = $\{\tau_1 (1, 0, 2, 6), \tau_2 (2, 0, 5, 9)\}$

Example Task Set 2 = $\{\tau_1 (1, 0, 2, 4), \tau_2 (2, 0, 4, 7)\}$

✂ The priority of the task is set according to the EDF scheduling rules.

✂ If there are tasks with the same deadlines, the task with a **lower task ID** will be executed first.

The input file format:

Task ID	Arrive Time	Execution Time	Task Periodic
##	##	##	##

Example of file 1:

```
1 0 2 6
2 0 5 9
```

Example output file of task set 1:

2	Completion	task(1)(0)	task(2)(0)	2	0	4
7	Completion	task(2)(0)	task(1)(1)	7	2	2
9	Completion	task(1)(1)	task(2)(1)	3	1	3
12	Preemption	task(2)(1)	task(1)(2)			
14	Completion	task(1)(2)	task(2)(1)	2	0	4
16	Completion	task(2)(1)	task(63)	7	2	2
18	Preemption	task(63)	task(1)(3)			
20	Completion	task(1)(3)	task(2)(2)	2	0	4
25	Completion	task(2)(2)	task(1)(4)	7	2	2
27	Completion	task(1)(4)	task(2)(3)	3	1	3
30	Preemption	task(2)(3)	task(1)(5)			
32	Completion	task(1)(5)	task(2)(3)	2	0	4
34	Completion	task(2)(3)	task(63)	7	2	2
36	Preemption	task(63)	task(1)(6)			
38	Completion	task(1)(6)	task(2)(4)	2	0	4

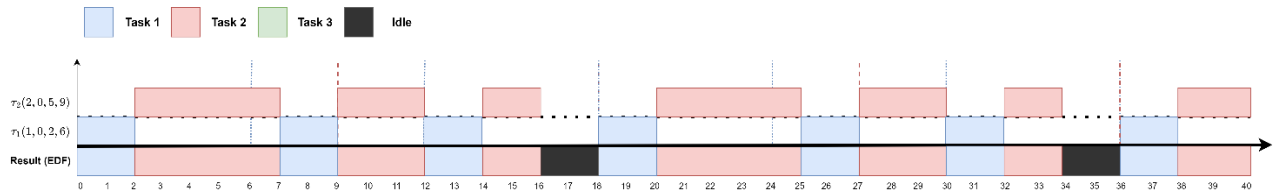
Evaluation:

The output format:

Tick	Event	CurrentTask ID	NextTask ID	Response Time	Preemption Time	OSTimeDly
##	Preemption	task(ID)(job number)	task(ID)(job number)			
##	Completion	task(ID)(job number)	task(ID)(job number)	##	##	##
##	MissDeadline	task(ID)(job number)	-----			

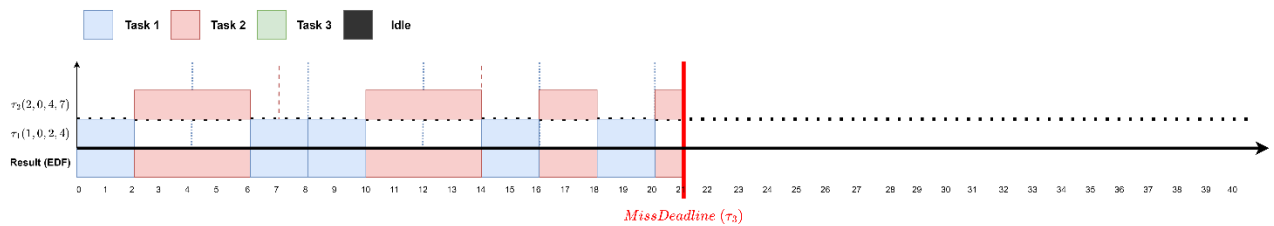
✖ If the task is Idle Task, print “*task(priority)*”.

The output results of **Example 1**:



Tick	Event	CurrentTask ID	NextTask ID	ResponseTime	PreemptionTime	OSTimeDly
2	Completion	task(1)(0)	task(2)(0)	2	0	4
7	Completion	task(2)(0)	task(1)(1)	7	2	2
9	Completion	task(1)(1)	task(2)(1)	3	1	3
12	Preemption	task(2)(1)	task(1)(2)			
14	Completion	task(1)(2)	task(2)(1)	2	0	4
16	Completion	task(2)(1)	task(63)	7	2	2
18	Preemption	task(63)	task(1)(3)			
20	Completion	task(1)(3)	task(2)(2)	2	0	4
25	Completion	task(2)(2)	task(1)(4)	7	2	2
27	Completion	task(1)(4)	task(2)(3)	3	1	3
30	Preemption	task(2)(3)	task(1)(5)			
32	Completion	task(1)(5)	task(2)(3)	2	0	4
34	Completion	task(2)(3)	task(63)	7	2	2
36	Preemption	task(63)	task(1)(6)			
38	Completion	task(1)(6)	task(2)(4)	2	0	4

The output results of **Example 2**:



Tick	Event	CurrentTask ID	NextTask ID	ResponseTime	PreemptionTime	OSTimeDly
2	Completion	task(1)(0)	task(2)(0)	2	0	2
6	Completion	task(2)(0)	task(1)(1)	6	2	1
8	Completion	task(1)(1)	task(1)(2)	4	2	0
10	Completion	task(1)(2)	task(2)(1)	2	0	2
14	Completion	task(2)(1)	task(1)(3)	7	3	0
16	Completion	task(1)(3)	task(1)(4)	4	2	0
18	Completion	task(1)(4)	task(2)(2)	2	0	2
21	MissDeadline	task(2)(2)	-----			

[Part II] CUS Scheduler Implementation

Objective:

To implement Constant Utilization Servers (CUS) for serving aperiodic tasks and to observe the scheduling behaviors.

Problem Definition:

As you did in Part I, uC/OS-II supports the EDF scheduling algorithm. Based on your EDF scheduler, you are going to implement Constant Utilization Servers (CUS) to serve aperiodic tasks.

Implement the following two task sets. Add necessary code to the μ C/OS-II scheduler **in the kernel level** to observe how the task suffers the schedule delay.

The following two examples include both periodic tasks and aperiodic jobs.

Periodic Task Set = $\{\tau_{ID} (ID, \text{arrival time}, \text{execution time}, \text{period})\}$

Aperiodic Job Set = $\{j_{num} (\text{num}, \text{arrival time}, \text{execution time}, \text{absolute deadline})\}$

===== Example 1 =====

Periodic Task Set = $\{\tau_1 (1, 0, 2, 8), \tau_2 (2, 0, 3, 10), \tau_3 (3, 0, 4, 15), \tau_4_ServerSize (4, 25\%)\}$

Aperiodic Jobs Set = $\{j_0 (0, 12, 3, 24), j_1 (1, 14, 2, 32)\}$

===== Example 2 =====

Periodic Task Set = $\{\tau_1 (1, 0, 2, 4), \tau_2 (2, 0, 3, 9), \tau_3_ServerSize (3, 30\%)\}$

Aperiodic Jobs Set = $\{j_0 (0, 1, 3, 24), j_1 (1, 11, 2, 39)\}$

===== Example 3 =====

Periodic Task Set = $\{\tau_1 (1, 0, 3, 6), \tau_2 (2, 0, 4, 10), \tau_3_ServerSize (3, 10\%)\}$

Aperiodic Jobs Set = $\{j_0 (0, 7, 1, 23), j_1 (1, 10, 2, 25)\}$

✂ The priority of a task is set according to the EDF scheduling rules.

✂ If there are tasks with the same deadlines, the task with a **lower task ID** will be executed first.

✂ If periodic job and aperiodic job with the same deadlines, **the periodic job** will be executed first.

Evaluation:

The additional output format for an aperiodic job:

Tick	
##	<i>if arrive time < sever deadline:</i> Aperiodic job (job number) arrives. Do nothing. Aperiodic job (job number) sets CUS server's deadline as ##.
	<i>else:</i> Aperiodic job (job number) arrives and sets CUS server's deadline as ##.
##	<i>if absolute Deadline < sever deadline:</i> Aperiodic job(job number) arrives. But can't Scheduled. (miss absolute deadline)
##	Aperiodic job (job number) is finished.

✂ The time tick of setting the server's deadline is according to the CUS scheduling rules.

The TaskSet.txt format:

Type	Task ID	Arrive Time	Execution Time	Task Periodic
Periodic	##	##	##	##
Type	Server ID	Server Size		
Server Size	##	##		

```
1 0 2 8
2 0 3 10
3 0 4 15
4 25
```

The AperiodicJobs.txt format:

Jobs No.	Arrive Time	Execution Time	Absolute Deadline
##	##	##	##

```
0 12 3 24
1 14 2 32
```

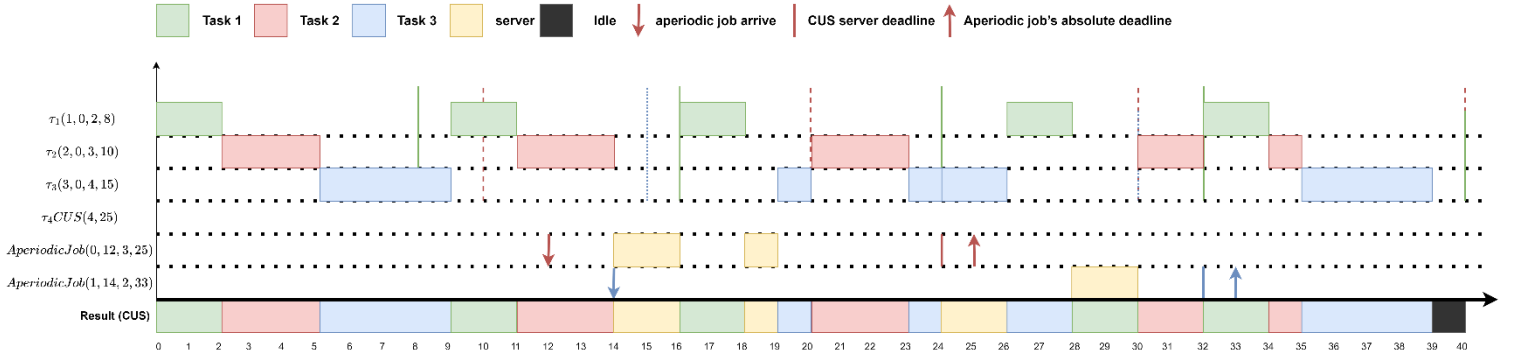
The output format:

Tick	Event	CurrentTask ID	NextTask ID	Response Time	Preemption Time	OSTimeDly
##	Preemption	task(ID)(job number)	task(ID)(job number)			
##	Completion	task(ID)(job number)	task(ID)(job number)	##	##	## or N/A
##	MissDeadline	task(ID)(job number)	-----			

Example output file of task set 1:

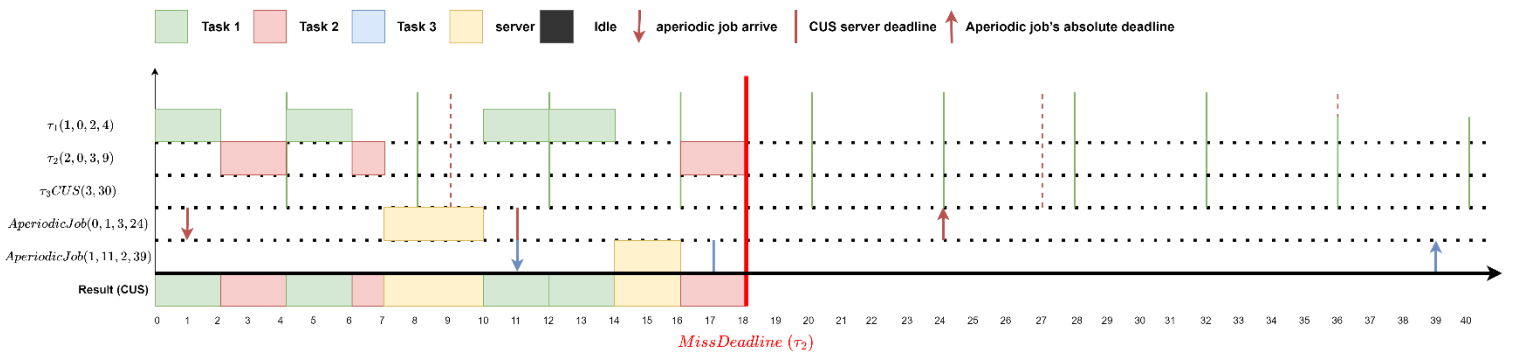
```
0      Completion    task( 1)( 0)    task( 2)( 0)    2      0      6
5      Completion    task( 2)( 0)    task( 3)( 0)    5      2      5
9      Completion    task( 3)( 0)    task( 1)( 1)    9      5      6
11     Completion    task( 1)( 1)    task( 2)( 1)    3      1      5
12     Aperiodic job(0) arrives and sets CUS server's deadline as 24.
14     Completion    task( 2)( 1)    task( 4)( 0)    4      1      6
14     Aperiodic job(1) arrives. Do nothing.
16     Preemption    task( 4)( 0)    task( 1)( 2)
18     Completion    task( 1)( 2)    task( 4)( 0)    2      0      6
19     Aperiodic job(0) is finished.
19     Completion    task( 4)( 0)    task( 3)( 1)    7      4      N/A
20     Preemption    task( 3)( 1)    task( 2)( 2)
23     Completion    task( 2)( 2)    task( 3)( 1)    3      0      7
24     Aperiodic job(1) sets CUS server's deadline as 32.
26     Completion    task( 3)( 1)    task( 1)( 3)    11     7      4
28     Completion    task( 1)( 3)    task( 4)( 1)    4      2      4
30     Aperiodic job(1) is finished.
30     Completion    task( 4)( 1)    task( 2)( 3)    6      4      N/A
32     Preemption    task( 2)( 3)    task( 1)( 4)
34     Completion    task( 1)( 4)    task( 2)( 3)    2      0      6
35     Completion    task( 2)( 3)    task( 3)( 2)    5      2      5
39     Completion    task( 3)( 2)    task(63)        9      5      6
40     Preemption    task(63)        task( 1)( 5)
```

The output results of **Example 1**:



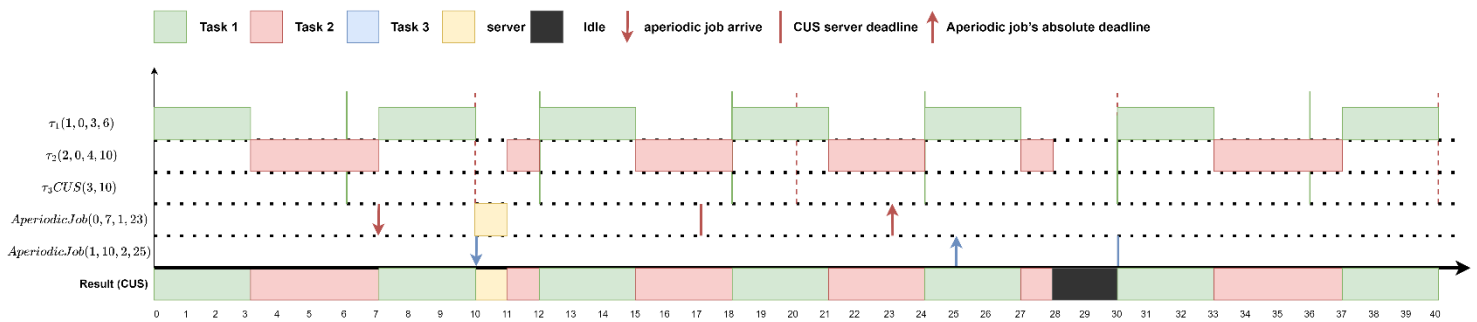
Tick	Event	CurrentTask ID	NextTask ID	ResponseTime	PreemptionTime	OSTimeDly
2	Completion	task(1)(0)	task(2)(0)	2	0	6
5	Completion	task(2)(0)	task(3)(0)	5	2	5
9	Completion	task(3)(0)	task(1)(1)	9	5	6
11	Completion	task(1)(1)	task(2)(1)	3	1	5
12	Aperiodic job(0) arrives and sets CUS server's deadline as 24.					
14	Completion	task(2)(1)	task(4)(0)	4	1	6
14	Aperiodic job(1) arrives. Do nothing.					
16	Preemption	task(4)(0)	task(1)(2)			
18	Completion	task(1)(2)	task(4)(0)	2	0	6
19	Aperiodic job(0) is finished.					
19	Completion	task(4)(0)	task(3)(1)	7	4	N/A
20	Preemption	task(3)(1)	task(2)(2)			
23	Completion	task(2)(2)	task(3)(1)	3	0	7
24	Aperiodic job(1) sets CUS server's deadline as 32.					
26	Completion	task(3)(1)	task(1)(3)	11	7	4
28	Completion	task(1)(3)	task(4)(1)	4	2	4
30	Aperiodic job(1) is finished.					
30	Completion	task(4)(1)	task(2)(3)	6	4	N/A
32	Preemption	task(2)(3)	task(1)(4)			
34	Completion	task(1)(4)	task(2)(3)	2	0	6
35	Completion	task(2)(3)	task(3)(2)	5	2	5
39	Completion	task(3)(2)	task(63)	9	5	6
40	Preemption	task(63)	task(1)(5)			

The output results of **Example 2**:



Tick	Event	CurrentTask ID	NextTask ID	ResponseTime	PreemptionTime	OSTimeDly
1	Aperiodic job(0) arrives and sets CUS server's deadline as 11.					
2	Completion	task(1)(0)	task(2)(0)	2	0	2
4	Preemption	task(2)(0)	task(1)(1)			
6	Completion	task(1)(1)	task(2)(0)	2	0	2
7	Completion	task(2)(0)	task(3)(0)	7	4	2
10	Aperiodic job(0) is finished.					
10	Completion	task(3)(0)	task(1)(2)	9	6	N/A
11	Aperiodic job(1) arrives and sets CUS server's deadline as 17.					
12	Completion	task(1)(2)	task1(1)(3)	4	2	0
14	Completion	task(1)(3)	task(3)(1)	2	0	2
16	Aperiodic job(1) is finished.					
16	Completion	task(3)(1)	task(2)(1)	5	3	N/A
18	MissDeadline	task(2)(1)	-----			

The output results of **Example 3**:



Tick	Event	CurrentTask ID	NextTask ID	ResponseTime	PreemptionTime	OSTimeDly
3	Completion	task(1)(0)	task(2)(0)	3	0	3
7	Completion	task(2)(0)	task(1)(1)	7	3	3
7	Aperiodic job(0) arrives and sets CUS server's deadline as 17.					
10	Completion	task(1)(1)	task(3)(0)	4	1	2
10	Aperiodic job(1) arrives. But can't scheduled. (miss absolute deadline)					
11	Aperiodic job(0) is finished.					
11	Completion	task(3)(0)	task(2)(1)	4	3	N/A
12	Preemption	task(2)(1)	task(1)(2)			
15	Completion	task(1)(2)	task(2)(1)	3	0	3
18	Completion	task(2)(1)	task(1)(3)	8	4	2
21	Completion	task(1)(3)	task(2)(2)	3	0	3
24	Preemption	task(2)(2)	task(1)(4)			
27	Completion	task(1)(4)	task(2)(2)	3	0	3
28	Completion	task(2)(2)	task(63)	8	4	2
30	Preemption	task(63)	task(1)(5)			
33	Completion	task(1)(5)	task(2)(3)	3	0	3
37	Completion	task(2)(3)	task(1)(6)	7	3	3
40	Completion	task(1)(6)	task(2)(4)	4	1	2

Credit:

[PART I] EDF Scheduler Implementation [50%]

- The correctness of schedule results of examples. Note the testing task set **might not** be the same as the given example task set. (30%)
 - Include and implement missed deadline handling in EDF.
- A report that describes your implementation (please attach the screenshot of the code and **MARK** the modified part). (10%)

[PART II] CUS Scheduler Implementation [50%]

- The correctness of schedule results of examples. Note the testing task set **might not** be the same as the given example task set. (40%)
 - Include and implement missed deadline handling in CUS.
- A report that describes your implementation (please attach the screenshot of the code and **MARK** the modified part). (10%)

[BOUNDS] CUS & Button-triggered Aperiodic Job [10%]

- Implement the CUS scheduling and set button-triggered events as aperiodic jobs. (10%)

※ You must modify the source code!

※ Standard input and output filenames in the project are necessary for the checker. Please check the file names before submitting. You must print out the result on the Output.txt file.

```
#define INPUT_FILE_NAME "./TaskSet.txt"
#define OUTPUT_FILE_NAME "./Output.txt"
#define APERIODIC_FILE_NAME "./Aperiodicjobs.txt"
```

※ Please set the system end time as **40** seconds in this project.

```
#define SYSTEM_END_TIME 40
```

※ You must check your project can produce the correct output file.

※ We will use **different task sets** to verify your code.

※ You will submit **two µC/OS-II projects** for PART I and PART II, respectively.

Project submit:

Submit to Moodle.

Submit deadline: May 8, 2025 (Thursday) 8:00

File name format: RTOS_Myyyddxxx_PA2.zip

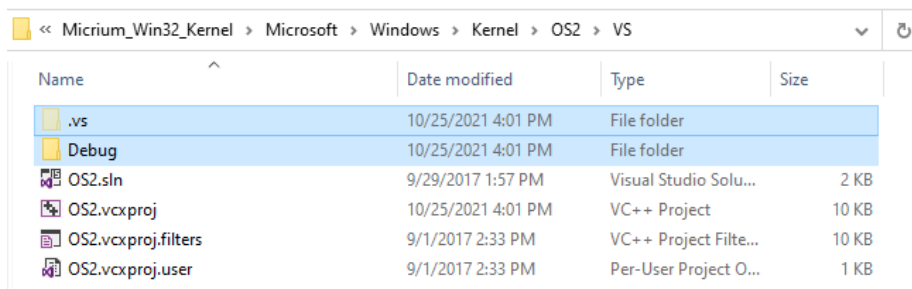
RTOS_Myyyddxxx_PA2.zip includes:

- The report (RTOS_Myyyddxxx_PA2.pdf).
- Folder with the executable EDF project (RTOS_Myyyddxxx_PA2_EDF).
- Folder with the executable CUS project (RTOS_Myyyddxxx_PA2_CUS).

※ Plagiarizing is strictly prohibited.

Hints:

1. Please delete the “.vs” and “Debug” folders.



<< Micrium_Win32_Kernel > Microsoft > Windows > Kernel > OS2 > VS				
Name	Date modified	Type	Size	
.vs	10/25/2021 4:01 PM	File folder		
Debug	10/25/2021 4:01 PM	File folder		
OS2.sln	9/29/2017 1:57 PM	Visual Studio Solu...	2 KB	
OS2.vcxproj	10/25/2021 4:01 PM	VC++ Project	10 KB	
OS2.vcxproj.filters	9/1/2017 2:33 PM	VC++ Project Filte...	10 KB	
OS2.vcxproj.user	9/1/2017 2:33 PM	Per-User Project O...	1 KB	

2. RTOS_Myyyddxxx_PA2.zip must be including files as follow:

